

Introduction to Web APIs

Prof. Dr. Peter Braun

19. November 2024

1 Titlepage

Introduction to Web APIs

Prof. Dr. Peter Braun



No Text – maybe some music :-)

2 Learning Goals

Learning Goals

- You learn about different API paradigms
- You learn JAX-RS to implement Web APIs
- You learn how to use the Quarkus Java framework

Welcome to today's unit! In this unit, we're diving into Web API development. After you have learned about the HTTP protocol, which is the driver of Web API, we will now explore how to develop Web APIs in a modern way in Java. You saw the concept of Java Servlets in one of the last units, but this was mentioned more for historical reasons and to understand how things work under the hood. First, we'll summarize the different API paradigms you'll likely encounter in real life, focusing on how these paradigms shape how systems communicate. From there, we'll delve into JAX-RS, a powerful and widely used API for building Java web services. Finally, you'll learn how to efficiently leverage the Quarkus framework to implement these web APIs. Quarkus is designed to create high-performance, cloud-native Java applications and integrates seamlessly with JAX-RS. Let's get started!

3 API Paradigms

API Paradigms

- Application Programming Interface
- RPC is based on **procedures** or **functions**
- Google RPC or GraphQL

- Web APIs are based on **documents** or **resources**
- Web APIs only provide standard operations

- REST APIs are a sub-set of Web APIs

The interface between a client and a server is called an API. API is the abbreviation for Application Programming Interface, and you know this term from your previous work as a software developer in various fields. For example, the public functions of a software component are also called API. However, here and in the future, we will always refer to API as the interface in a client-server system. As usual in a client-server system, the server makes services available for a client. We summarize the collection of functions and the data structures the service provides under the term "API of the server." Various styles have evolved over the last few decades in the design of such APIs. In the previous units, you have learned of remote procedure calls, abbreviated as RPC. In this technique, the server provides its interface as a collection of procedures. A function can be considered a function or method in today's programming language, such as C++ or Java. The function has a name, parameters are passed when the function is called, and the function responds with a result. Because this technique is so close to the experience of most programmers, it is still used in many distributed systems today. For the software developer, using such an API is very easy because its philosophy fits very well with the other thinking of a software developer in procedural programming languages. But here in these units, we are dealing with the HTTP protocol and the World Wide Web or the philosophy behind the architecture of the World Wide Web. And in this environment, a fundamentally different way of thinking about client-server interfaces has evolved. The basic principle of the World Wide Web is based on the idea that a server provides documents that clients can access or modify. Such a document can be, for example, a web page, which would be the simplest case. But it can also be an image, an audio recording, or a video. However, we can also talk more abstractly about the data provided by the server. Such data can, for example, come from a database and be provided in a structured data format such

as JSON or XML. In this approach, a document can be either a single or finite set of records. We will first start with the concept of Web APIs. We will also talk about REST APIs, a subset of Web API. Some very strict requirements must be fulfilled when you want to provide REST APIs; unfortunately, this makes your life as a developer a bit difficult.

4 Designing an API based on Documents

Designing an API based on Documents

- **Base URL:** `https://example.org/api`
- **POST** on `/universities` creates a University
- **GET** on `/universities` requests all Universities
- **GET** on `/universities/1` requests University with id 1
- **PUT** on `/universities/1` updates University with id 1
- **DELETE** on `/universities/1` deletes University with id 1
- **A Web API only provides these four HTTP verbs**

The transfer protocol in a web-based system used between client and server is HTTP, and we assume that you already have a basic understanding of this protocol. The HTTP protocol consists of messages sent between the client and the server. A request sent from the client to the server always contains one of the HTTP verbs as an operation or command. There is only a very small list of allowed HTTP verbs, and on the slide beside me, you can see the four most important ones. The example is about managing resources of type universities. That means we need the four simple CRUD operations for universities first. The service we are discussing here could be hosted on a server named `example.org`. The first path element in the URL is “API,” which is also very common in real life. The data about universities is then available under the sub-path element “universities.” We start with the ‘POST’ verb. When you send a POST request to ‘/universities,’ a new University resource is created. This is how we handle resource creation in Web APIs. If you have some experience in Web development, you will know that POST is used to send data from the client to the server, usually the contents of input fields. Next is ‘GET’. A GET request to ‘/universities’ retrieves all universities, while a GET to ‘/universities/1’ fetches the specific University with the ID of 1. We use ‘PUT’ to update a resource. For example, sending a PUT request to ‘/universities/1’ updates the existing University with ID 1. Finally, ‘DELETE’ is used to remove resources. A DELETE request to ‘/universities/1’ deletes the University with ID 1. It’s important to emphasize that these four HTTP verbs—POST, GET, PUT, and DELETE—are sufficient to handle the entire lifecycle of your resources in a Web API. These operations allow you to create, read, update, and delete data, providing a complete and efficient way to manage data. You might wonder why it is sufficient to have only four operations here. The answer is that in a Web API, we deal with documents and not with functions. Triggering operations

are done by manipulating documents.

5 JAX-RS

JAX-RS

- Jakarta RESTful Web Services version 4 (April 2024)
- Uses Java annotations for almost all configurations
- RESTEasy (from Redhat; integrated into Quarkus)
- Jersey (from Eclipse; has even more features)
- Spring is not compatible (but frequently used)

In one of the last units, you learned how to implement Java servlets. This technology can also be used to implement Web APIs, but it would require a lot of work. Let's now look at how to implement such a Web API on the server more easily in the Java programming language. For this purpose, Java provides us with a specification called JAX-RS, which is now Jakarta RESTful Web Services. This specification defines some basic classes, mostly interfaces, and numerous Java annotations. The specification is to be separated from the implementation, and therefore, we need another library before we can use JAX-RS in our projects. This is the same as with the JPA specification and Hibernate as a provider. We use RESTEasy, one of the two reference implementations of this specification, because it integrates nicely with Quarkus, a very nice framework that I want you to learn. Jersey is another possible implementation of JAX-RS. Jersey is even better than RESTEasy because it provides more features, but many are outside the scope of the JAX-RS specification.

6 Screencast

Screencast

- Show how to clone and start the demo app

When you clone the given repository it should look like this. This is the package structure here. It contains simple implementation of the hexagonal architecture. You can see the domain logic and the ports here. You can see the out adapter for persistence here. And you can see the in adapter for the web API here. Please read this file, the README file, and read the source code. But don't copy the source code without questioning. It is just one implementation. It's not the or the best, by far not, implementation of this architecture and of this, let's say, functionality. In this first screencast I would like to show you three ways to work with this backend. And the first way is how to start the development mode of QVACOS. And you can see the command here. It's just maven compile qvacos-dev. And we go to the console and we type this in. And this should look on your computer very, very similar. So now we are here in the, let's say, interactive mode of QVACOS. QVACOS is now running and I can demonstrate that it works when we use postman. This is one of the URLs that the application supports. And when we send a GET request to this URL, you can see here the result, a list of universities. QVACOS now is able to do hot reloading. And I will demonstrate this using this log message here, loading all universities. And we go to the source code and look for this log message. I have a log message here. And we go to the source code and look for this log message. I think it's implemented in the service. So here. And I modify the log message. And I only have to save this now. And we can immediately start the request again. And Maven does, QVACOS does everything that is needed to exchange or replace the class file that has changed. You can see here the updated log message. So it's not necessary to shut down everything, compile, package, restart and so on. You can just let QVACOS run, change in your backend code. And when you call, when you request the URLs the next time, everything is reloaded. You can also, you can also activate here to run the test

cases. You can see here in the log message. That was, it was rerun. And the test cases are implemented here. There are two kinds of test cases here implemented for demonstration. This test case is the one that would be executed when QVACOS is started in DEV mode. And this class here with this IT at the end, this is the integration test. And how this can be started I will show you later. So you can type H here for more options. So you can handle how to run tests here. You can restart the system from time to time, very rare. It is necessary that you restart the system. So in this case just type letter S. You don't have to quit Maven here and restart, just type letter S. You can also toggle live reload here. Okay, with Q you quit everything here. And the second way I would like to demonstrate is to start integration testing. And this is done using Docker. So you can also take the Docker, the POM file here, POM Docker. So I have two Maven files, POM for the normal execution and POM for Docker. You can read this file here to understand how Docker can be configured in a Maven file. So executing this command starts the, or creates a Docker file, starts the container and let the integration tests run against this container. So first the normal tests are executed here. And now the Docker image is created. Now Quarkus has started. And that was successful. So here you can see this integration test was executed. And the third way to start the application is to create the Docker image and start the container by yourself. So we start building the image here. And now we start the container. And the port that we open is port 8080. And we can go to Postman again and we start the request. And here you can see the result.

7 Handling GET Requests

Handling GET Requests

```
1 @Path("/universities")
2 public class UniversityService
3 {
4     @GET
5     @Produces("application/json")
6     public Response getUniversities() {
7         List<University> allUniversities = ...; // retrieve from database
8         return Response.ok( allUniversities ).build();
9     }
10
11     @Path("/{id}")
12     @GET
13     @Produces("application/json")
14     public Response getUniversityById( @PathParam("id") long id ) {
15         University university = ...; // retrieve from database
16         return Response.ok( university ).build();
17     }
18 }
```

■ Method `ok` produces status code 200

I want to show you some code snippets to implement HTTP message handling in RESTEasy. There will be a separate method for each HTTP verb, and all these methods belong to a class named “UniversityService.” In line 1, we use the annotation `Path` to define the path under which this service should be offered. If the hostname is “example.org” and the first path element is like “api” – that would be defined somewhere else, we add another path element to the necessary URL. The full path that the client must then use is now “/api/persons.” The first method, ‘getUniversities’, is annotated with ‘GET’ and ‘Produces(“application/json”)’. This indicates that it handles HTTP GET requests and returns a list of universities in JSON format. The method retrieves all universities from the database and uses ‘Response.ok’ in line 8 to wrap the list in an HTTP response. The ‘ok’ method automatically sets the HTTP status code 200, which signals a successful operation. The second method, ‘getUniversityById’, also uses ‘GET’ and ‘Produces(“application/json”)’, but it’s additionally annotated with ‘Path(“id”)’ in line 11. This means it handles GET requests for a specific university, identified by the ‘id’ path parameter. The ‘PathParam(“id”)’ annotation maps the value from the URL to the ‘id’ parameter in the method. After retrieving the corresponding university from the database, the method uses ‘Response.ok’ to return the result again with a 200 status code.

8 Handling POST Requests

Handling POST Requests

■ More methods in class `UniversityService`:

```
1 @POST
2 @Consumes("application/json")
3 public Response createUniversity(University data)
4 {
5     // Create a new University in the database
6     return Response.created( ).build();
7 }
```

■ The request body is mapped to `UniversityData`

■ Method `created` produces status code 201

Let's take a closer look at implementing a POST method using JAX-RS. Here, we have a method annotated with 'POST', which indicates that it handles HTTP POST requests. Additionally, the 'Consumes("application/json")' annotation specifies that this method expects the request body to be in JSON format. The method signature includes a 'University' parameter, which means the JSON request body will be automatically mapped to a 'University' object. JAX-RS does this mapping, leveraging the framework's JSON deserialization capabilities. Now, let's talk about the response. The method returns a 'Response' object, and it uses the 'Response.created()' method, which is specifically designed to signal the successful creation of a resource. This method automatically sets the response's HTTP status code 201—"Created"—.

9 Handling PUT Requests

Handling PUT Requests

■ More methods in class `UniversityService`:

```
1 @PUT
2 @Path("{id}")
3 @Consumes("application/json")
4 public Response updateUniversity( @PathParam("id") long id,
5                                  University data )
6 {
7     //Send update request to database
8     return Response.status( HttpServletResponse.NO_CONTENT.code() )
9         .build();
10 }
```

■ The method responds with status code 204

■ The response body is empty by intention

Now, let's examine the implementation of a PUT method in JAX-RS. This method is annotated with 'PUT,' indicating that it handles HTTP PUT requests, typically used to update existing resources. The 'Path("id")' annotation specifies that this method operates on a specific resource, identified by the 'id' path parameter. The '@PathParam("id")' annotation maps this value from the URL to the 'id' parameter in the method. The method also uses 'Consumes(application/json)', indicating that it expects a JSON request body, which is deserialized into a 'University' object. You might wonder why providing an ID and updated data is necessary. The JSON representation must contain the resource's ID anyway. However, this is common practice because the URL must identify the object uniquely. In the response, we return a 'Response' object with a status code of 204 (No Content). This is achieved using 'Response.status' with the constant value NO_CONTENT,' which stands for 204. The 204 status code signals that the update was successful but intentionally includes an empty response body, as no additional content is required in this case. This is a common pattern when the server successfully processes an update request but doesn't need to return any specific data to the client.

10 Handling DELETE Requests

Handling DELETE Requests

■ More methods in class `UniversityService`:

```
1 @DELETE
2 @Path("{id}")
3 public Response deleteUniversity( @PathParam("id") long id )
4 {
5     // Delete the given resource in the database
6     return Response.status( HttpServletResponse.NO_CONTENT.code() )
7         .build();
8 }
```

■ The method responds with status code 204

■ The response body is empty by intention

Let's now explore the implementation of a DELETE method in JAX-RS. This method is annotated with 'DELETE', indicating that it handles HTTP DELETE requests, which are used to remove resources from the server. The 'Path("{id}")' annotation specifies that this method operates on a specific resource identified by the 'id' path parameter. The '@PathParam(id)' annotation maps this value from the URL to the 'id' parameter in the method. When a resource is successfully deleted, the method returns a 'Response' object with the status code 204 again. As with the previous example, the 204 status code indicates that the operation was successful, and the response body is intentionally empty. This approach is a standard practice when the server doesn't need to provide additional information after successfully deleting a resource.

11 Screencast

Screencast

- Show how the demo works with Postman

I want to demonstrate how this backend can be used with Postman. First I start Quarkus. The four HTTP verbs that I showed on the slides are implemented in this class here, University Web Controller. The implementation here is of course now complete in a sense that it works. And we go to Postman and I start the first request. And you can see the JSON formatted result here. The next request is the one that goes to single entities or single resources. And you add the ID here and the result looks like this. Next we want to create a new university. We send a post request here and now we have to fill the body of the request message. And we pass these three parameters here. And the result is 201 created. And you can see in the response there is a location header that goes to this newly created resource. And this looks strange here because I wanted to demonstrate how this validation mechanism works. That is implemented in the backend. So when we pass here an empty string for university city, the backend will respond with a bad request. And will tell you that the city name must not be blank. And this is implemented in the backend. Here this is the DTO object for the interface. And here we have this annotation not blank for all these three object attributes. Now we want to, or we can request this new university here. So this was the content. And now we can delete this. 204, this has worked. Now we send a get request to this URL. And we get a 404 not found. And finally we can send a put request. We also have to give here the correct representation of the university. And in this case it has worked. Be aware that these two IDs here must be identical. Because if we change this, we get a bad request. And the error message is not so good in this case. But it says that the path is not so good. And we can also change the path. And the error message is not so good in this case. But it says that the path ID is 2. And the resource ID was 1000. And this is not allowed. So I think that's it. That are the four typical HTTP verbs in five use cases. Get collection, get single, post, put,

delete. Please read the source code. And in the next units we are going to extend this source code step by step.

12 Summary

Summary

- You learned about document-oriented APIs
- JAX-RS provides an easy technique for Web APIs

That's it for the first introduction to JAX-RS. In this session, you explored how to build document-oriented APIs. These APIs rely on structured data formats like JSON and XML to communicate between the client and server, making it easy to work with modern web applications. We demonstrated how JAX-RS simplifies the development of Web APIs using Java annotations. With just a few lines of code, you can define endpoints for common HTTP methods like POST, PUT, and DELETE and handle typical use cases such as resource creation, updates, and deletions. JAX-RS is a powerful tool that abstracts much of the complexity of RESTful service implementation, enabling you to focus on building robust, maintainable web applications.

13 Literature

Literature

- Bill Burke: RESTful Java with JAX-RS 2.0: Designing and Developing Distributed Web Services (Englisch). O'Reilly, 2013.